LambdaMap: Serverless Dynamism on Big Data Processing

Steven Golob, Austin Bomhold, Simarpal Singh
School of Engineering and Technology
University of Washington Tacoma

ABSTRACT

In this study, we present a MapReduce architecture implemented using AWS Lambda, where a master Lambda function orchestrates the invocation of N worker Lambda functions to efficiently distribute computational tasks. We evaluate this architecture's performance through two applications: a word count application and an iterative k-means clustering application. These applications are compared against their implementations in PySpark, executed within Docker containers on AWS Fargate services, and on variously-sized EC2 instances. Our comparative analysis focuses on key metrics such as cost, runtime, and memory usage, providing insights into the effectiveness and efficiency of MapReduce on a serverless paradigm versus traditional PySpark implementations.

1 Introduction

Serverless computing is becoming an increasingly-used cloud computing platform, and is leveraged by major companies such as Netflix and Airbnb. It is one of the most recent and highest-level cloud abstractions that allows applications to be broken down into microservices. The advantages to this, from a development perspective, are that project maintenance becomes more easily delegated and less coupled. But from a resource-availability perspective, it stands out in its ability to scale. AWS Lambda, for example—a function-as-a-service platform that provides serverless infrastructure—has the massive hardware resources to allow microservices to *horizontally* scale dynamically and nimbly.

Therefore, A smaller company would benefit from making their services available as microservices in AWS Lambda for several reasons. For one, the company would not need to predict customer usage upfront, and could leave the scaling abstraction to the serverless platform. Also, they would not need to pay an upfront cost for hardware that may be under-utilized, and could simply pay proportional to their usage instead. Lastly, the microservices are easier to manage, requiring less configuration from the company, allowing it to keep applications highly flexible and simple.

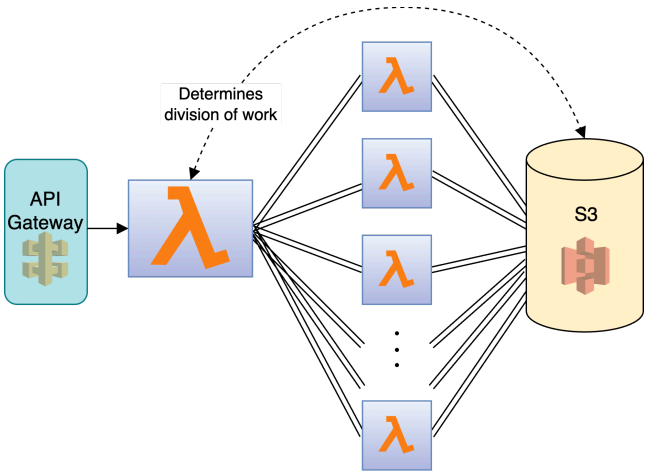

Figure 1: **LambdaMap uses a "master" AWS Lambda function to divide a big data processing task among *N* "worker" Lambda functions, which each collect their portion of the data from AWS S3.**

In this work, we leverage the power of serverless computing on a different use case: instead of studying performance for small jobs, we look at how it can perform big data processing tasks. We do this by creating an alternative to MapReduce processing, *LambdaMap*. Specifically, LambdaMap creates a "master" AWS Lambda function (operating similarly to the master node in MapReduce) that delegates the work to *N* "worker" Lambda functions. The worker functions then individually gather their portion of the data that needs to be processed from AWS Simple Storage Service (S3), perform their computations, and then send their result back to the master. The master then aggregates these results into the final result, which can be sent to S3 or AWS Simple Notification Service (SNS). *N* is specified by the client at runtime, depending on runtime or cost-efficiency imperatives. Our architecture is visualized in Figure 1.

This approach to big data processing is similar to those of MapReduce frameworks like Apache Spark. However, the key difference in LambdaMap is that worker nodes are serverless functions, and are therefore dynamic and can use an arbitrary number of workers.

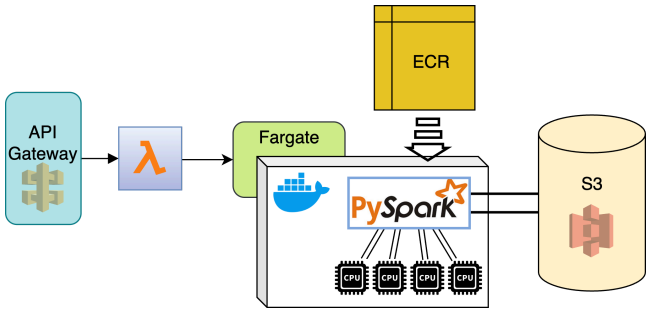

Figure 2: **Comparison #1: running Apache PySpark on AWS Fargate.**

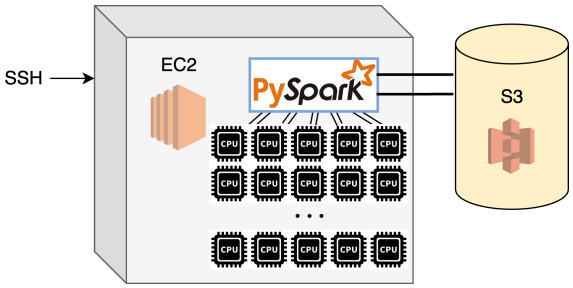

Figure 3: **Comparison #2: running Apache PySpark on EC2 instances with various vCPU amounts.**

In our investigation, we analyze LambdaMap's scalability, runtime, and overall costs. We do so by running LambdaMap on two disparate big data processing jobs. The first one is representative of a compute-bound task: a simple word count over a large amount of text files. We use the Project Gutenberg[1] repository of books as our data. We use 1,000 files, each ranging from 80 bytes to 6 megabytes, totalling 345 megabytes in all.

The second application is performing K-Means clustering. We synthetically generated 920 files to use for the task, each with 10,000 points in 20-dimensional euclidean space. This workload totals 3.31 gigabytes. Additionally, we require that the clustering is performed over *25 iterations*. So this second application is much resource-intensive, and is representative of a MapReduce job that requires far more coordination steps between the master and the workers.

To look into the benefits of using LambdaMap, we compare its performance on these two applications against the performance of the same in Apache PySpark. We run Apache PySpark in a docker container in two settings: First, the container is pulled from AWS ECR, and then run on AWS Fargate, which has 16 vCPUs, under two memory settings. This is invoked via an API Gateway endpoint (Shown in Figure 2). Second, the container is simply run on compute-optimized AWS EC2 instances with between 4 and 192 vCPU configurations (Shown in Figure 3, and Table 1). All code in LambdaMap and the Apache PySpark code that runs in Docker is implemented in Python, and can be found at https://github.com/steveng9/MapReduceInLambda.

1.1 Implementation of master and worker functions

Currently, LambdaMap is not set up as a generalized framework, meaning that the word count and K-Means clustering applications are hard-coded into versions of LambdaMap. It would not be difficult to make it generalizable in a similar way to how Apache Spark generalizes the *map* and *reduce* stages into its API. However, the purpose of this work is not to yet provide such a framework, but to develop a proof of concept of serverless big data processing and to investigate its performance. On the other hand, these implementations are generalized such that the number of desired worker functions N is passed to the master, through API Gateway, at runtime.

For word count, the master function simply retrieves a list of the names of the files needing to be processed from S3, as well as their sizes. The master node does not retrieve the files themselves. Then the master node makes N lists of file names, where the total size in bytes of all the files in each list is *roughly* even. This is done by quickly sorting the sizes of the files, and assigning to each list in a round. A `ThreadPoolExecutor` is then used launch N worker nodes via the `boto3` library, sending a different list of names to each worker node as a parameter[2]. The worker nodes each receive their assigned file names, retrieve them from S3, and perform a count, send the counts back to the master, which then aggregates the N sets of counts. Note that the master stays idle while the workers work. We assign the workers each **2,000 MB** of memory, to be assured at least one full vCPU share [1], and the master is given **8,500 MB** of memory, assuring it at least 4 vCPU shares to handle the hundreds of threads of worker invocations.

In the K-Means application, the master must call all N workers *for each iteration*. We specify 25 iterations, as is generally accepted for K-Means. This represents applications where MapReduce may rely upon several back-and-forth coordinating steps between the worker nodes and the master.

[1] https://www.gutenberg.org/

[2] This sorting mechanism is imperfect, given that the file sizes are variable. But our goal is to efficiently assign work. Assigning even workloads more robustly is left for future work.

2 Experiments and results

2.1 Setup

We run LambdaMap, and Apache PySpark on the word count application and the K-Means clustering application. For the word count, we compute over 1,000 text files from the Project Gutenberg repository, each between 80 bytes and 6 megabytes, totalling 345 megabytes. For K-Means clustering, we synthetically generate 3.31 gigabytes of 20-dimensional points. We distribute them across 920 files, each containing 10,000 points. We conduct 25 iterations for K-Means, and set K = 15.

Importantly for each experiment, we make sure the *N*+1 sandboxes required to run LambdaMap are awake and pre-warmed before testing. We verify this in the inspections conducted by SAAF,[3] a profiling framework that we use to collect metrics. All results are averaged over two trials.

2.2 Tradeoffs in runtime

Shown in Figure 4 are the runtimes of LambdaMap on the word count and clustering applications. As expected, the overall runtime generally drops when more workers are used. For word count, we tried up to 900 workers with diminishing returns in efficiency. With less than 85 workers, the master function hit the maximum Lambda function time of 15 minutes, and timed out. For K-Means, the master timed out when employing less than 75 workers, and also could not execute successfully for more than 450 workers. This is perhaps because, over 25 iterations, there is a great deal of overhead in launching (450 x 25) lambda functions, and each of them reading ⌈920 / 450⌉ files from S3.

Figure 5 shows the same applications running in docker containers, in variously sized EC2 instances, as well as running on the 16 vCPU Fargate service. We used the instance types specified in Table 1 to get the required vCPU count.

vCPUs	instance	memory (GB)	on demand cost / hr ($)
4	c5d.xlarge	8	0.1920
16	c5d.4xlarge	32	0.7680
36	c5d.9xlarge	72	1.7280
64	c6id.16xlarge	128	3.2256
192	c6a.48xlarge	384	7.3440

Table 1: **Specifications of EC2 instances used in experiments.**

[3] https://github.com/wlloyduw/SAAF

In the graph, notice that for word count, Fargate with 120 GB of memory has the quickest runtime when using 16 vCPUs, and is slightly slower with only 32 GB. In K-Means, the runtimes for 120 GB, 32 GB Fargate, and EC2 are virtually identical when 16 vCPUs are used.

On these datasets, the runtime improvements taper off in the EC2 instance when using more than 32 vCPUs. Using 192 CPUs appears to be wasteful, and even results in a slightly longer runtime for word count than when using 64 vCPUs.

K-Means clustering takes far longer than the word count because of the larger dataset size, and because of the more complex coordination happening between the master and worker nodes in PySpark.

We attempted to compare these architectures against a serial, single-threaded implementation of these applications. However, neither single-threaded implementation for word count, nor for K-Means, terminated after two days, presumably because of the strain on memory caused by the datasets. We consider running on a single vCPU in large-memory EC2s to be unjustified, and so did not try this.

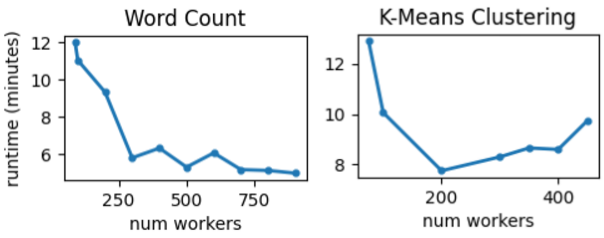

Figure 4: **Runtimes of the word count and K-Means applications in LambdaMap, for various numbers of workers *N*.**

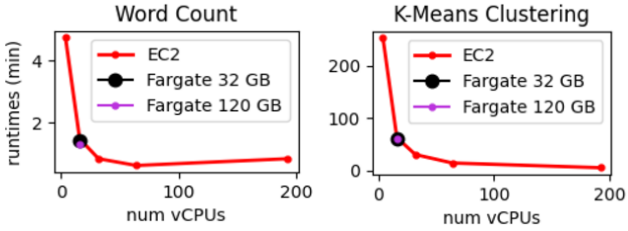

Figure 5: **Runtimes of the word count and K-Means applications in EC2 containers, and in a 32GB and 120GB Fargate Docker container with 16 vCPUs.**

# worker	(s)	# worker	(s)	# worker	(s)
1	.035	16	2.39	200	31.1
2	.146	32	4.77	300	46.0
4	.480	64	9.12	400	61.2
8	1.15	100	14.9	800	121.7

Table 2: **Total runtime in seconds to invoke, and wait for the responses of *N* worker Lambda functions, when using the `ThreadPoolExecutor`.**

2.3 LambdaMap's overhead

In Table 2, we present the runtimes of the master Lambda when simply instantiating *empty* worker Lambdas. All function sandboxes are pre-warmed, and all Lambdas are in the same region. When instantiating one worker, the spin-up cost, the overhead of the `ThreadPoolExecutor`, plus the round-trip time, is 35 milliseconds. When spinning up and managing 8 functions, the overhead goes up to 1.15 seconds—far more than proportionally. Merely spinning up 800 empty lambda functions, and receiving their response, takes over two minutes. This high cost begins to resemble a proportional relationship with the number of workers for higher numbers, suggesting that the master needs more resources, or there could be a bottleneck in the `ThreadPoolExecutor`. These overhead costs become important when the job requires more master-worker coordination and iteration, such as in the case of K-Means clustering.

2.3 Tradeoffs in cost

Costs for LambdaMap are calculated using

$$C_\lambda = 0.0000166667 \cdot (8.5 r_m + 2.0 \sum_{w \in \mathcal{W}} r_w)$$

where r_m is the runtime for the master lambda, and r_w is the runtime of each worker lambda w in $\mathcal{W}$. The master and worker are allocated 8.5 GB and 2.0 GB, respectively. The cost of each 16 vCPU Fargate job is calculated using

$$C_\mathcal{F} = r \cdot (16 \cdot 0.04048 + g \cdot 0.004445)$$

with g either 32 GB or 120 GB, and r as its runtime in seconds. On-demand and spot-request EC2 costs are simply calculated by their instance cost multiplied by the runtime in hours. Using these calculations, we plot the runtime of each experiment, and the associated cost in Figure 6. In the appendix, we provide exact costs for each experiment in Tables 3, 4 and 5.

As these plots show, LambdaMap is more costly for our applications than running PySpark in a container. For word count, it also is slower, and so has little benefit. Across the configurations, runtimes didn't vary significantly with different workforces. This is especially the case for K-Means, where one can see the blue points (unironically) clustering around 10 minutes and costing $1.40. However, in K-Means, LambdaMap is faster than all containers, except the EC2 instance with 192 vCPUs. Actually, the 192 vCPU EC2 finishes the job so fast that it is *cheaper* than LambdaMap as well, even though it costs $7.34 per hour (on demand).

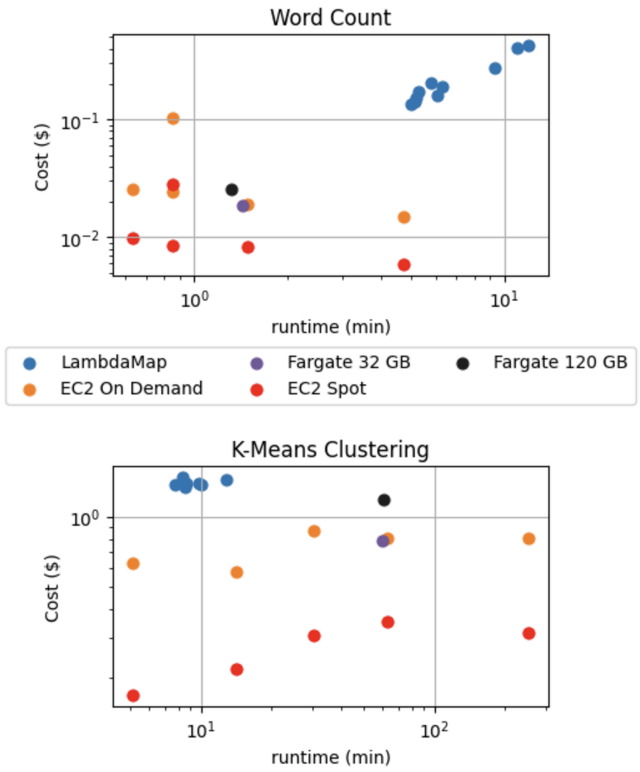

Figure 6: **Costs of all experiments, for each architecture, against the total runtimes. Both the x- and y-axis are in log scale.**

3 Discussion

The use of an EC2 instance to execute these word count and K-Means applications is the apparent choice, given these results. It can execute the jobs as quickly as LambdaMap if needed, and costs less. The disadvantage of this approach is that the EC2 needs to be manually spun up and shut-down promptly to avoid greater costs. Fargate did not offer an improvement in costs or runtime, given the same vCPU allocation as the EC2.

And of course, LambdaMap underperformed on our hopes. Its runtimes did not vary greatly for different amounts of workers. For example, all runtimes were

between 8 and 13 minutes for the K-Means job, and orchestrating more than 450 workers crashed our application.

Lambda's time-out of 15 minutes is certainly a limitation of LambdaMap. Choosing *N* depends on investigating how many workers are needed to complete the job within the time limit, since the optimal *N* varies by task, as we've seen in these experiments. Interestingly enough, despite the substantial overhead and cost of running *N* worker Lambdas, LambdaMap seems to become *more* cost effective as *N* increases. We see this in Figure 6 for word counting, where hiring more workers results in the quickest runtimes *and* less overall cost.

Another counter-intuitive result was that, on K-Means, LambdaMap was faster than any EC2 or Fargate experiment (except for when the EC2 had 192 vCPUs) for any amount of worker lambdas, between 75 and 450. We had expected the opposite because of the overhead in invoking *N* worker Lambdas for 25 iterations. But ostensibly the analogous overhead in PySpark is also high.

4 Related work and future work

BabelMR was proposed in 2023 to distribute workloads written in the MapReduce paradigm across serverless functions [2]. However, its purpose is to lower barriers to MapReduce, emphasizing its generalizability. Ours, on the other hand, focusses on two very specific compute-bound applications, and shows a more comprehensive cost breakdown. Ray provides a similar service that is oriented towards ease-of-use, but underperforms in runtime [3, 2]. PyWren provides serverless MapReduce in Python, but is severely slower than BabelMR [4, 2]. Mahling et al. study PySpark via clusters, and find that the overhead of cluster management is significant [2]. And Corral is yet another serverless MapReduce framework that has difficulty with usability and handling larger workloads [5].

And finally, AWS provides the Elastic MapReduce (EMR) service.[4] We leave comparing LambdaMap's cost and runtime against EMR for future work. There are also other interesting dimensions to this study that we leave for future work, such as making LambdaMap's master function asynchronous instead of having to wait idle for the worker functions to finish, causing double-billing. Also, while in this work, the master and worker lambda functions' memory sizes remain fixed, we hope to investigate the performance of master and worker functions with other memory (and thus vCPU) allocations. Furthermore, the work-delegation algorithm in the master function for word count and K-Means is naive, and may lead to workers receiving mildly different workloads. More sophisticated delegation algorithms could be studied. Most exciting of all would be to run LambdaMap on much larger workloads, requiring tens of thousands of worker functions to complete within 15 minutes.

[4] https://aws.amazon.com/emr/

ACKNOWLEDGMENTS

Writing a portion of the AWS configuration code in Python, as well as refining various parts of the text, were assisted by ChatGPT. The skills learned in following Professor Wes Lloyd's tutorials were highly beneficial in building LambdaMap. Additionally, he granted us $100 in AWS cloud-computing credits, aptly, since $82.04 of which was used in developing and testing this project.

REFERENCES

[1] Cordingly, Robert, Sonia Xu, and Wes Lloyd. "Function memory optimization for heterogeneous serverless platforms with cpu time accounting." *2022 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2022.

[2] Mahling, Fabian, Paul Rößler, Thomas Bodner, and Tilmann Rabl. "BabelMR: A Polyglot Framework for Serverless MapReduce." In *VLDB Workshops*. 2023.

[3] Anyscale, Inc., Ray, https://www.ray.io/, 2023.

[4] Jonas, Eric, Qifan Pu, Shivaram Venkataraman, Ion Stoica, and Benjamin Recht. "Occupy the cloud: Distributed computing for the 99%." In *Proceedings of the 2017 symposium on cloud computing*, pp. 445-451. 2017.

[5] Congdon, Ben. "Corral: A serverless MapReduce framework written for AWS Lambda", https:// github.com/bcongdon/corral/, 2023.

APPENDIX

Word Count				K-Means Clustering			
# vCPU	OD cost ($)	Spot cost ($)	(min)	# vCPU	OD cost ($)	Spot cost ($)	(min)
4	0.02	0.01	4.72	4	0.81	0.31	253
16	0.02	0.01	1.49	16	0.80	0.35	62.8
36	0.02	0.01	0.85	36	0.87	0.30	30.3
64	0.03	0.01	0.63	64	0.58	0.22	14.1
192	0.01	0.03	0.85	192	0.63	0.17	5.12

Table 3: **Costs and runtimes for EC2 instances.**

Word Count			K-Means Clustering		
# workers	cost ($)	(min)	# workers	cost ($)	(min)
90	0.42	12.0	75	1.45	12.9
100	0.41	11.0	100	1.38	10.1
200	0.27	9.34	200	1.37	7.75
300	0.20	5.80	300	1.48	8.30
400	0.19	6.33	350	1.40	8.66
500	0.17	5.31	400	1.35	8.60
600	0.16	6.07	450	1.39	9.74
700	0.15	5.17			
800	0.14	5.13			
900	0.14	4.98			

Table 4: **Costs and runtimes for LambdaMap on both applications, for all *N* workers tried.**

Word Count				K-Means Clustering			
# vCPU	32 GB cost ($)	120 GB cost ($)	ave. time (min)	# vCPU	32 GB cost ($)	120 GB cost ($)	ave. time (min)
16	0.02	0.03	1.37	16	0.79	1.18	60.2

Table 5: **Costs and runtimes for Fargate containers.**